OrbitLLM: Profile-Anchored Large-Model Scheduling for LEO/NTN Mobile Edge Systems

Anonymous Author(s)
Affiliation
City, Country
email@domain

Abstract

Emerging 6G non-terrestrial networks (NTNs) make LEO satellites intermittent mobile-edge nodes, but current offloading models still treat workloads as generic bits or CPU cycles. This abstraction is brittle for large language models (LLMs): autoregressive inference has a key–value cache memory wall, and prefill and decode tokens have different energy costs. We present **OrbitLLM**, a profile-anchored scheduling framework that exposes LLM energy, latency, and memory surfaces to an online LEO/NTN edge scheduler. Rather than treating our host as spacecraft hardware, we profile 1.7–8B Qwen-family models on an RTX 4090 24GB availability anchor and make the profile table replaceable. Given visibility windows, value half-lives, battery budget, downlink capacity, and memory limits, OrbitLLM chooses among full on-board inference, raw downlink, and lightweight pre-screening with partial downlink. A mixed-integer program gives an offline upper bound, while Heuristic-TVD implements a constant-time value-density rule. In 24-hour simulations with 10 random seeds, Heuristic-TVD reaches 0.90 ± 0.01 normalized value, above All-Downlink (0.59 ± 0.01), All-On-Board (0.75 ± 0.01), and Fixed-Threshold (0.85 ± 0.01). Additional network-contact, pre-screen, and hardware-scaling sweeps show that the scheduler adapts its ON/PRE/DOWN action mix as NTN resources and profile costs change.

Keywords

LEO/NTN edge computing, large language models, on-board inference, energy-aware offloading, KV cache

1 Introduction

LEO satellites are becoming part of the mobile Internet architecture rather than isolated space relays. In emerging 6G non-terrestrial networks (NTNs), satellites may serve remote sensors, direct-to-device links, and delay-sensitive mobile services during intermittent contacts. At the same time, sensing pipelines are moving from narrow convolutional models toward large language and vision–language models [6, 11, 14]. The natural next step is to process high-value observations at the orbital edge instead of always waiting for a ground-station pass.

The scheduling literature is not yet ready for this step. Most satellite-edge offloading formulations model a task by its input bits, CPU cycles, or service delay [5, 8, 17]. That view misses two facts that dominate LLM inference. First, peak memory grows with context through the KV cache; a schedule that ignores this boundary can select an on-board action that simply does not fit. Second, prefill and decode tokens have different energy and throughput behavior because the former is compute-heavy and the latter is memory-bandwidth-heavy [7, 9]. A scheduler that collapses them into one average cost misprices long generations, exactly the case expected in open-ended Earth-observation reports.

This paper develops OrbitLLM as a workshop-scale scheduling layer for LEO/NTN mobile edge systems. We do not attempt to reproduce a full orbital hardware campaign. Instead, we build a measured LLM cost surface from RTX 4090 availability-anchor profiles and study how such a replaceable profile changes the on-board/downlink decision. Each task has a data size, required model scale, and exponentially decaying value. The policy selects one of three architecture actions: run the model on board, downlink the raw data for ground inference, or run a lightweight semantic pre-screen and downlink only retained evidence.

Our contributions are:

- **LLM-aware cost model.** We expose prefill energy, decode energy, throughput, and peak memory as explicit functions of model scale, quantization, and context length.
- **Energy–value formulation.** We combine the LLM cost surface with orbital visibility windows, battery limits, downlink capacity, and time-decaying task values.
- **Lightweight scheduler.** We give a MILP upper bound and a deployable heuristic, Heuristic-TVD, that greedily maximizes value density while enforcing hard energy and memory feasibility.
- **Reproducible simulation.** We release a Python pipeline that generates task traces, runs baselines, performs network/PRE/hardware sweeps, writes CSV results, and regenerates every plot and table.

2 Related Work

2.1 NTN and Satellite Edge Computing

LEO systems are increasingly studied as part of the mobile/NTN edge rather than only as bent-pipe relays. Recent work optimizes delay–cost trade-offs for emergency tasks [8], adaptive offloading across heterogeneous constellations [17], hierarchical resource allocation [5], and deep-RL user association or service migration [15, 16]. These formulations are valuable, but their workload model is typically a divisible amount of bits or CPU cycles. OrbitLLM keeps the orbital visibility and resource-allocation view, but replaces the generic compute abstraction with LLM-specific memory and token-energy constraints.

2.2 Mobile Edge AI and Pre-Screening

Space-AI systems have shown that compact neural models can run near the sensor. Reviews summarize hardware and software support for satellite inference [4, 10], while systems such as Kodan show the benefit of selecting valuable data before expensive processing or downlink [3]. Remote-sensing foundation models and VLM datasets are rapidly improving the capability side [6, 13, 14]. OrbitLLM treats pre-screening as an architecture action: a lightweight module retains semantic evidence under a configurable bit-retention and utility-retention curve.

2.3 Efficient LLM Inference

Efficient LLM inference provides the mechanisms that make on-board deployment plausible: quantization, KV-cache compression, small language models, and memory-aware serving [2, 12, 18]. KV-cache management is especially relevant because it determines whether long-context inference fits a constrained accelerator [1, 7, 9]. OrbitLLM does not propose a new compression method; it treats the measured or parameterized effect of those methods as the cost surface used by the scheduler.

3 System Model and Problem Formulation

3.1 Scenario

We consider one LEO/NTN edge node equipped with a sensor-facing task queue, a constrained accelerator, and an intermittent downlink. As shown in Figure 1, tasks enter an on-board queue. A replaceable LLM profile supplies energy, latency, and memory costs; an orbital contact schedule supplies ground-station windows; the scheduler selects an action for each task.

3.2 Visibility and Tasks

The horizon is discretized into time slots $t = 1, \dots, T$ of duration Δ. The orbital schedule gives a visibility indicator

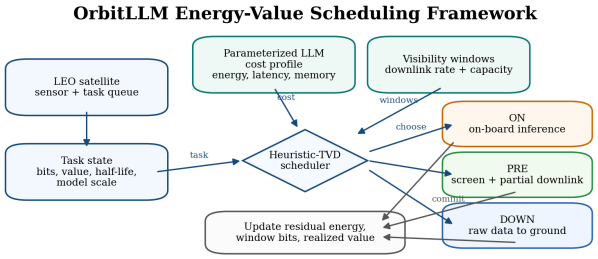

Figure 1: OrbitLLM architecture. A profile of LLM inference costs and a visibility-window timeline feed an online energy–value scheduler that chooses on-board inference, direct downlink, or pre-screening with partial downlink.

v_t and rate r_t when $v_t = 1$. For a visibility window w, the available downlink capacity is

$$B_w = \sum_{t \in w} r_t \Delta. \tag{1}$$

Task i arrives at time a_i and is described by $(d_i, s_i, V_i^0, \tau_i, n_i^p, n_i^d)$: raw bits, required model scale, immediate value, value half-life, prefill tokens, and decode tokens. If completed at time c_i, the realized value is

$$V_i(c_i) = V_i^0 \exp\left(-\frac{c_i - a_i}{\tau_i}\right), \qquad c_i \geq a_i. \tag{2}$$

Small τ_i models time-critical observations such as wildfire onset; large τ_i models archival mapping.

3.3 Actions and LLM Costs

The scheduler chooses $x_i \in \{\text{ON}, \text{DOWN}, \text{PRE}\}$. ON runs the full model on board; DOWN transmits raw data; PRE is a lightweight semantic compression action inspired by value-aware data selection [3]. It can be implemented by an ROI selector, confidence-gated detector, or small encoder that emits metadata plus selected patches. We model it by a retained-bit ratio ρ_i and a utility-retention factor η_i: it transmits $\rho_i d_i$ bits and realizes $\eta_i V_i(c_i)$. These parameters are not hidden constants; Section 6 sweeps them to show when PRE is useful. For model scale s_i, quantization q, and context $n_i^p + n_i^d$, the profile gives prefill energy e^p, decode energy e^d, throughput, and peak memory. Full on-board energy is

$$E_i^{\text{ON}} = e_{s_i,q}^p n_i^p + e_{s_i,q}^d n_i^d. \tag{3}$$

The pre-screen action uses a configurable fraction of this energy and memory and is evaluated across multiple ρ_i, η_i regimes.

Table 1: INT4 profile rows used by the reported simulation at 2048-token context. Energy is prefill/decode mJ per token, averaged over three runs.

Model	Quant.	Tok/s	Energy/tok	Peak mem
1.7B	INT4	25.4	2.4/3787.2	2.9
3B	INT4	22.7	5.2/4668.7	4.1
8B	INT4	20.2	16.0/7018.6	7.7

3.4 Optimization Problem

The objective is to maximize cumulative value subject to battery, downlink, and memory feasibility:

$$\max_{\{x_i,\rho_i\}} \quad \sum_i V_i(c_i(x_i)) \tag{4}$$

$$\text{s.t.} \quad \sum_{i:x_i=\text{ON}} E_i^{\text{ON}} + \sum_{i:x_i=\text{PRE}} E_i^{\text{PRE}} \le E^{\max}, \tag{5}$$

$$\sum_{i\in w} b_i(x_i) \le B_w, \quad \forall w, \tag{6}$$

$$\text{peakmem}(s_i, q, n_i^p + n_i^d) \le M^{\max}. \tag{7}$$

Here $b_i(\text{DOWN}) = d_i$, $b_i(\text{PRE}) = \rho_i d_i$, and $b_i(\text{ON}) = 0$; PRE values are multiplied by η_i. Even with known arrivals this contains a multiple-choice knapsack problem; the MILP in Section 5 is therefore an offline upper bound rather than an on-board algorithm.

4 Parameterized LLM Cost Model

4.1 Profile Interface

OrbitLLM uses a profile table rather than a generic CPU-cycle cost. Each row contains model name, scale, quantization, context length, prefill tokens, decode tokens, prefill joules/token, decode joules/token, decode throughput, peak memory, and GPU power. We generated this table on a remote RTX 4090 24GB host using local copies of `Qwen3-1.7B`, `Qwen2.5-3B-Instruct`, and `Qwen3-8B`. The benchmark covers FP16, INT8, and INT4 inference at 1024, 2048, and 4096-token contexts with three runs per point; repeated runs are averaged before simulation. Replacing `results/profile_anchor.csv` with another measured CSV automatically regenerates all scheduling results.

4.2 Energy and Memory Surfaces

Figure 2 shows the measured separation between prefill and decode energy. The distinction matters because a long generation consumes more energy than a single averaged token coefficient would predict. Figure 3 shows peak memory over model scale and context length. This surface is used as a hard feasibility check in Eq. (7); if a task exceeds the memory ceiling, it cannot be assigned to ON.

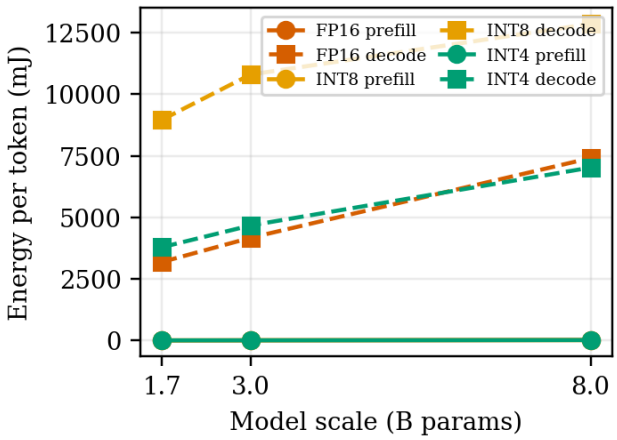

Figure 2: Prefill and decode energy per token at 2048-token context. The scheduler indexes this surface when computing Eq. (3).

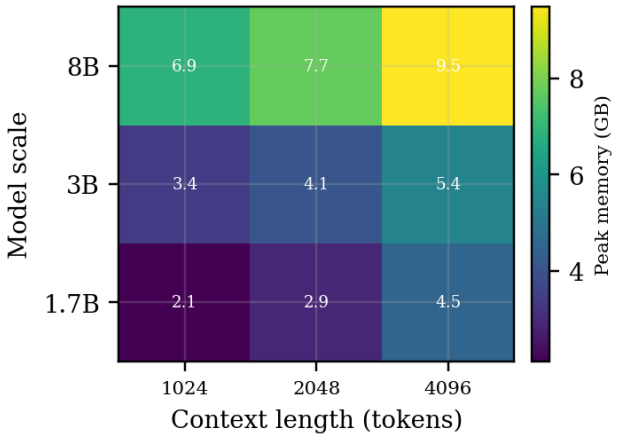

Figure 3: Peak memory for INT4 inference as a function of model scale and context length. The simulation uses a 16 GB on-board memory ceiling; all measured INT4 anchors fit below it.

4.3 Why Parameterization Helps

The purpose of this profile is not to claim flight-hardware performance. It is to make the scheduling assumption explicit and replaceable. Existing offloading models hide LLM behavior behind a scalar compute cost; OrbitLLM exposes the quantities that determine feasibility and value: memory fit, on-board latency, and token-level energy. The measured bitsandbytes INT8 path is not universally better than FP16 on this host, which reinforces the need to use hardware-specific profile rows rather than assumed quantization multipliers.

Algorithm 1: Heuristic-TVD Online Scheduling

Input: tasks, visibility windows, energy budget $E^{\max}$, memory ceiling $M^{\max}$, LLM profile Θ
Output: action x_i for each task
$E_{\text{res}} \leftarrow E^{\max}$; initialize downlink queue;
for *each task i in arrival order* **do**
  read $(e^p, e^d, \text{tok/s}, \text{peakmem})$ from Θ;
  enumerate $a \in \{\text{ON}, \text{DOWN}, \text{PRE}\}$;
  remove actions that exceed E_{res} or $M^{\max}$;
  estimate completion time, value, energy, and bits for each remaining action;
  $x_i \leftarrow \arg\max_a \text{TVD}_i(a)$;
  commit compute energy and downlink bits for x_i;

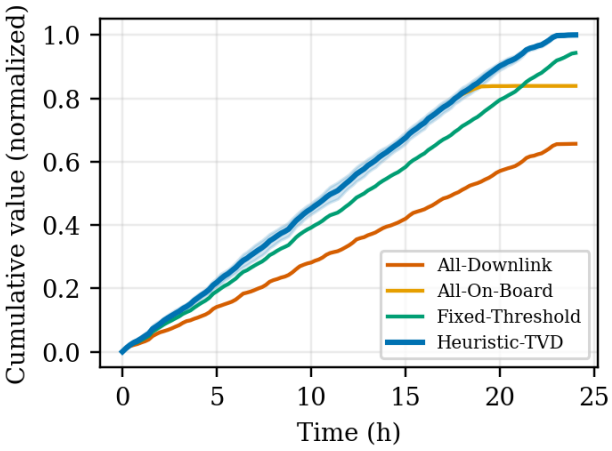

Figure 4: Cumulative realized value over 24 hours. Heuristic-TVD preserves energy for later high-value tasks while still serving urgent tasks early.

5 Scheduling Algorithms

5.1 MILP Upper Bound

With full knowledge of arrivals and windows, Eq. (4)–Eq. (7) can be solved as a mixed-integer linear program. We introduce binary variables y_i^a for action $a \in \{\text{ON}, \text{DOWN}, \text{PRE}\}$ and impose $\sum_a y_i^a = 1$. The energy constraint is global, the bandwidth constraint is per visibility window, and infeasible memory choices receive zero eligibility. The MILP is solved only offline on the ground and gives an upper bound for causal policies.

5.2 Heuristic-TVD

The deployable policy is a constant-time value-density rule. For every arriving task, it computes the feasible completion time and realized value for each action. Let $V_i(a)$ be the realized value of action a and let $V_i(\text{DOWN})$ be the direct-downlink reference. We score each feasible action by

$$\text{TVD}_i(a) = \frac{V_i(a) + \beta \max(0, V_i(a) - V_i(\text{DOWN}))}{\epsilon + \lambda_E E_i(a)/\max(E_{\text{res}}, 1) + \lambda_B b_i(a)/d_i}. \quad (8)$$

The numerator rewards both absolute value and value recovered relative to waiting for downlink; the denominator penalizes scarce energy and bandwidth. Actions violating memory or residual energy are removed before scoring.

5.3 Baselines

We compare against All-Downlink, which queues every raw observation for ground inference; All-On-Board, which runs on board until energy or memory prevents it; and Fixed-Threshold, which uses a static value threshold to decide whether to process or pre-screen on board. These baselines isolate the benefit of using time decay, LLM cost, and residual resources jointly.

6 Evaluation

6.1 Setup

The simulator runs a 24-hour horizon with 10 random seeds. Unless otherwise stated, traces use a 90-minute orbit, a 10-minute contact window, and a 35 Mbps downlink. Each trace contains 260 heterogeneous tasks with model scales in $\{1.7, 3, 8\}$B, log-normal data sizes, urgent and routine half-lives, and configured prefill/decode token counts. The on-board compute-energy budget is 150 kJ and the memory ceiling is 16 GB. The default PRE module uses 18% of full on-board inference energy/latency, retains 32% of raw bits, and keeps 94% of task utility; Figure 6 sweeps these assumptions. All numbers and plots are regenerated from CSV files produced on the remote RTX 4090 server.

6.2 Main Results

Figure 4 and Table 2 compare Heuristic-TVD with three baselines and the MILP upper bound. All-Downlink loses value because urgent tasks wait for contacts. All-On-Board has low latency but exhausts energy and drops tasks. Fixed-Threshold uses task value but cannot jointly adapt to residual energy, bandwidth, and model memory. Heuristic-TVD reaches 0.90 ± 0.01 normalized value, above All-Downlink (0.59 ± 0.01), All-On-Board (0.75 ± 0.01), and Fixed-Threshold (0.85 ± 0.01), while reporting zero OOM and zero energy violations.

6.3 NTN Contact Sensitivity

Figure 5 varies the network regime. At low downlink rates, Heuristic-TVD preserves value by avoiding raw transfers and using ON/PRE for urgent tasks. As contact windows

Table 2: Main 24-hour results over 10 seeds. Value is normalized by the MILP bound; bandwidth saving is relative to All-Downlink.

Method	Value↑	J/task↓	BW save↑	Lat. med.↓
All-Downlink	0.59±0.01	0.0	0.0	36.1
All-On-Board	0.75±0.01	760.7	100.0	1.1
Fixed-Threshold	0.85±0.01	355.1	64.4	6.4
Heuristic-TVD	0.90±0.01	585.2	84.9	1.0
MILP-Bound	1.00±0.00	576.9	–	–

Table 3: Action mix in the main experiment. PRE is selected only when partial semantic transfer is more valuable than raw downlink under residual resources.

Method	ON	PRE	DOWN	DROP
All-Downlink	0.0	0.0	100.0	0.0
All-On-Board	75.8	0.0	0.0	24.2
Fixed-Threshold	40.7	34.8	24.5	0.0
Heuristic-TVD	75.7	6.7	17.6	0.0

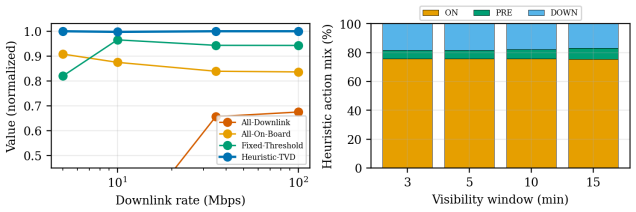

Figure 5: Network/contact sensitivity. Left: value across downlink rates. Right: Heuristic-TVD action mix as visibility windows change.

expand, the action mix shifts smoothly because the value-density denominator accounts for both residual energy and bandwidth. This is the key MobiArch point: the policy is an architecture-level adaptation layer, not a single fixed offloading threshold.

6.4 PRE and Hardware Sensitivity

Figure 6 sweeps PRE retained-bit ratio ρ and utility-retention model $\eta(\rho)$. PRE is most useful in the middle regime: aggressive filtering saves bandwidth but can lose value, while large ρ approaches raw downlink. This turns PRE from a black-box assumption into an explicit design space.

Figure 7 scales the measured 4090 energy and latency profile by 0.5–4×. Faster profiles choose full on-board inference; constrained profiles shift toward PRE and DOWN while preserving feasibility. Thus the framework does not rely on the 4090 as target hardware: it consumes a replaceable cost surface.

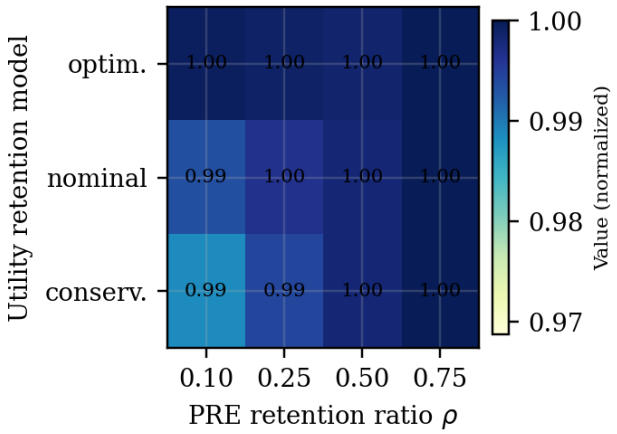

Figure 6: PRE sensitivity over retained bits and utility-retention regimes. Values are normalized against the best PRE/no-PRE choice for the same seed and setting.

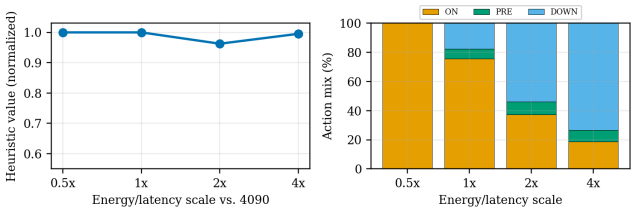

Figure 7: Hardware-profile scaling. Heuristic-TVD changes its action mix as the measured profile is scaled from faster accelerators to constrained edge devices.

7 Discussion and Limitations

OrbitLLM's main point is architectural: LEO/NTN mobile-edge schedulers need a runtime interface that exposes the quantities binding large-model inference. Bits and CPU cycles are insufficient when memory fit depends on KV cache length and energy depends on the prefill/decode mix. Making these quantities explicit also makes the framework reproducible: changing hardware, quantization, or model family only changes the profile CSV, not the optimizer or simulator.

The most important limitation is that the reported numbers are RTX 4090 availability-anchor measurements, not spacecraft accelerator measurements. The hardware-scaling experiment is intended to test sensitivity to this choice, not to validate a particular on-orbit device. Radiation effects, spacecraft integration, accuracy changes under quantization, and detailed power replenishment are outside the scope of this workshop version.

PRE is also an abstraction rather than a single deployed vision module. We model it as lightweight semantic compression with retained-bit and utility-retention parameters, then

sweep those parameters to expose when the action helps. A deployment would calibrate this curve with a concrete ROI selector, detector, or small vision-language encoder. Finally, our 24-hour traces are generated by a discrete-event simulator; future work should add real contact traces, multi-satellite interactions, and measured profiles from flight-relevant edge accelerators.

8 Conclusion

We presented OrbitLLM, a profile-anchored scheduling framework for large-model inference in LEO/NTN mobile edge systems. By turning LLM-specific inference behavior into a replaceable cost profile, the framework avoids the generic bits/cycles abstraction used by conventional offloading models. A simple value-density heuristic, calibrated against a MILP upper bound, outperforms All-Downlink, All-On-Board, and fixed-threshold baselines while respecting energy and memory constraints. Network, PRE, and hardware-scaling sweeps show how the ON/PRE/DOWN action mix adapts as mobile-edge resources change.

Reproducibility Statement

The artifact contains the benchmark CLI, simulator, MILP bound, plotting scripts, configuration file, generated CSV results, and LaTeX table macros. The code and results are available at https://github.com/1740919441a/OrbitLLM. Running `python -m orbitllm.cli simulate` followed by `python -m orbitllm.cli plot` regenerates the reported tables and figures.

References

[1] Keivan Alizadeh, Seyed Iman Mirzadeh, Dmitry Belenko, S. Karen Khatamifard, Minsik Cho, Carlo C. Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. 2024. LLM in a flash: Efficient Large Language Model Inference with Limited Memory. (2024), 12562–12584. doi:10.18653/v1/2024.acl-long.678

[2] Guangji Bai, Zheng Chai, Ling Chen, Shiyu Wang, Jiaying Lu, Nan Zhang, Tingwei Shi, Ziyang Yu, Mengdan Zhu, Yifei Zhang, Xinyuan Song, Carl Yang, Yue Cheng, and Liang Zhao. 2024. Beyond Efficiency: A Systematic Survey of Resource-Efficient Large Language Models. In *arXiv (Cornell University)*. Cornell University. doi:10.48550/arxiv.2401.00625

[3] Bradley Denby, Krishna Chintalapudi, Ranveer Chandra, Brandon Lucia, and Shadi A. Noghabi. 2023. Kodan: Addressing the Computational Bottleneck in Space. (2023), 392–403. doi:10.1145/3582016.3582043

[4] Lorenzo Diana and Pierpaolo Dini. 2024. Review on Hardware Devices and Software Techniques Enabling Neural Network Inference Onboard Satellites. *Remote Sensing* 16, 21 (2024), 3957. doi:10.3390/rs16213957

[5] Xiangqiang Gao, Yingmeng Hu, Yingzhao Shao, Hangyu Zhang, Yang Liu, Rongke Liu, and Jianhua Zhang. 2024. Hierarchical Dynamic Resource Allocation for Computation Offloading in LEO Satellite Networks. *IEEE Internet of Things Journal* 11, 11 (2024), 19470–19484. doi:10.1109/jiot.2024.3367937

[6] Chunlei Huo, Keming Chen, Shuaihao Zhang, Zeyu Wang, Heyu Yan, Jing Shen, Yu Hong, G. R. Qi, Hongmei Fang, and Zihan Wang. 2025. When Remote Sensing Meets Foundation Model: A Survey and Beyond. *Remote Sensing* 17, 2 (2025), 179. doi:10.3390/rs17020179

[7] Minsu Kim, Seongmin Hong, Ryeowook Ko, Soongyu Choi, Hunjong Lee, Junsoo Kim, Joo-Young Kim, and Jongse Park. 2025. Oaken: Fast and Efficient LLM Serving with Online-Offline Hybrid KV Cache Quantization. (2025), 482–497. doi:10.1145/3695053.3731019

[8] Changhao Li, Zhenmou Liu, Z. Q. Ye, Guoguang Wen, Zong-Fu Luo, and Chuanfu Zhang. 2025. Delay-cost computation offloading for on-board emergency tasks in LEO Satellite Edge Computing networks. *Future Generation Computer Systems* 169 (2025), 107797. doi:10.1016/j.future.2025.107797

[9] Haoyang Li, Yiming Li, Anxin Tian, Tianhao Tang, Zhao-Dong Xu, X Chen, Ning Hu, Wei Dong, Qing Li, and Lei Chen. 2024. A Survey on Large Language Model Acceleration based on KV Cache Management. In *arXiv (Cornell University)*. Cornell University. doi:10.48550/arxiv.2412.19442

[10] Cédric Léonard, Dirk Stober, and Martin Schulz. 2026. FPGA-Enabled Machine Learning Applications in Earth Observation: A Systematic Review. *Comput. Surveys* 58, 11 (2026), 1–36. doi:10.1145/3800686

[11] Gengchen Mai, Weiming Huang, Jin Sun, Suhang Song, Deepak R. Mishra, Ninghao Liu, Song Gao, Tianming Liu, Gao Cong, Yingjie Hu, Chris Cundy, Ziyuan Li, Rui Zhu, and Ni Lao. 2024. On the Opportunities and Challenges of Foundation Models for GeoAI (Vision Paper). *ACM Transactions on Spatial Algorithms and Systems* 10, 2 (2024), 1–46. doi:10.1145/3653070

[12] Fali Wang, Zhiwei Zhang, Xianren Zhang, Zongyu Wu, Tzuhao Mo, Qiuhao Lu, Wanjing Wang, Rui Li, Junjie Xu, Xianfeng Tang, Qi He, Yao Ma, Mingzhi Huang, and Suhang Wang. 2024. A Comprehensive Survey of Small Language Models in the Era of Large Language Models: Techniques, Enhancements, Applications, Collaboration with LLMs, and Trustworthiness. In *arXiv (Cornell University)*. Cornell University. doi:10.48550/arxiv.2411.03350

[13] Zhecheng Wang, Rajanie Prabha, Tianyuan Huang, Jiajun Wu, and Ram Rajagopal. 2024. SkyScript: A Large and Semantically Diverse Vision-Language Dataset for Remote Sensing. *Proceedings of the AAAI Conference on Artificial Intelligence* 38, 6 (2024), 5805–5813. doi:10.1609/aaai.v38i6.28393

[14] Xingxing Weng, Chao Pang, and Gui-Song Xia. 2025. Vision-Language Modeling Meets Remote Sensing: Models, datasets, and perspectives. *IEEE Geoscience and Remote Sensing Magazine* 13, 3 (2025), 276–323. doi:10.1109/mgrs.2025.3572702

[15] Haonan Wu, Xiumei Yang, and Zhiyong Bu. 2024. Task Offloading With Service Migration for Satellite Edge Computing: A Deep Reinforcement Learning Approach. *IEEE Access* 12 (2024), 25844–25856. doi:10.1109/access.2024.3367128

[16] Hangyu Zhang, Hongbo Zhao, Rongke Liu, Xiangqiang Gao, and Shenzhan Xu. 2024. Dynamic User Association and Computation Offloading in Satellite Edge Computing Networks via Deep Reinforcement Learning. *IEEE Transactions on Green Communications and Networking* 8, 4 (2024), 1888–1901. doi:10.1109/tgcn.2024.3357813

[17] Liang Zhao, M. Zeng, Ammar Hawbani, Yunhe Sun, Lexi Xu, Zhi Liu, and Xiaoming Zhou. 2025. A Region Division-Based Adaptive Task Offloading in Collaborative LEO Heterogeneous Constellation. *IEEE Internet of Things Journal* 12, 14 (2025), 26523–26537. doi:10.1109/jiot.2025.3559984

[18] Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, Luning Wang, Zhihang Yuan, Xiuhong Li, Shengen Yan, Guohao Dai, Xiaoping Zhang, Yuhan Dong, and Yu Wang. 2024. A Survey on Efficient Inference for Large Language Models. In *arXiv (Cornell University)*. Cornell University. doi:10.48550/arxiv.2404.14294